法律声明

- 本课件包括演示文稿、示例、代码、题库、视频和声音等内容，小象学院和主讲老师拥有完全知识产权的权利；只限于善意学习者在本课程使用，不得在课程范围外向任何第三方散播。任何其他人或机构不得盗版、复制、仿造其中的创意及内容，我们保留一切通过法律手段追究违反者的权利。

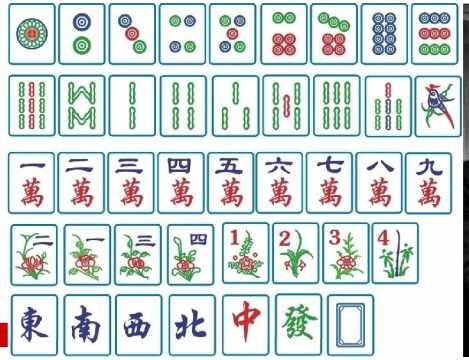
- 课程详情请咨询
  - 微信公众号：小象
  - 新浪微博：ChinaHadoop

概率组合算法

小象学院 ChinaHadoop.cn

邹博

身边的概率



- “国粹”麻将是集技巧与运气的在我国开展广泛的娱乐项目。去除花牌后的标准麻将，由1到9的“万、条、饼”各4张，以及“东南西北中发白”各4张，共计136张组成。我们把两张内容一样的牌叫一幅“将”。
- 请问，庄家起手摸14张牌，则他起手没有“将”的概率是多少？
  - 此外，可以算下摸13张牌没有“将”的概率，摸13张牌没有“风”的概率。

问题分析——古典概型

- 基本事件数目：
  - 一共136张牌，随意选4张，取法为：C_{136}^{14}
- 有效事件数目：
  - 一共34组牌，选择某个组，然后在该组的4张中任选1张，取法为：$C_{34}^{14} \cdot 4^{14}$
- “无将”的概率为：

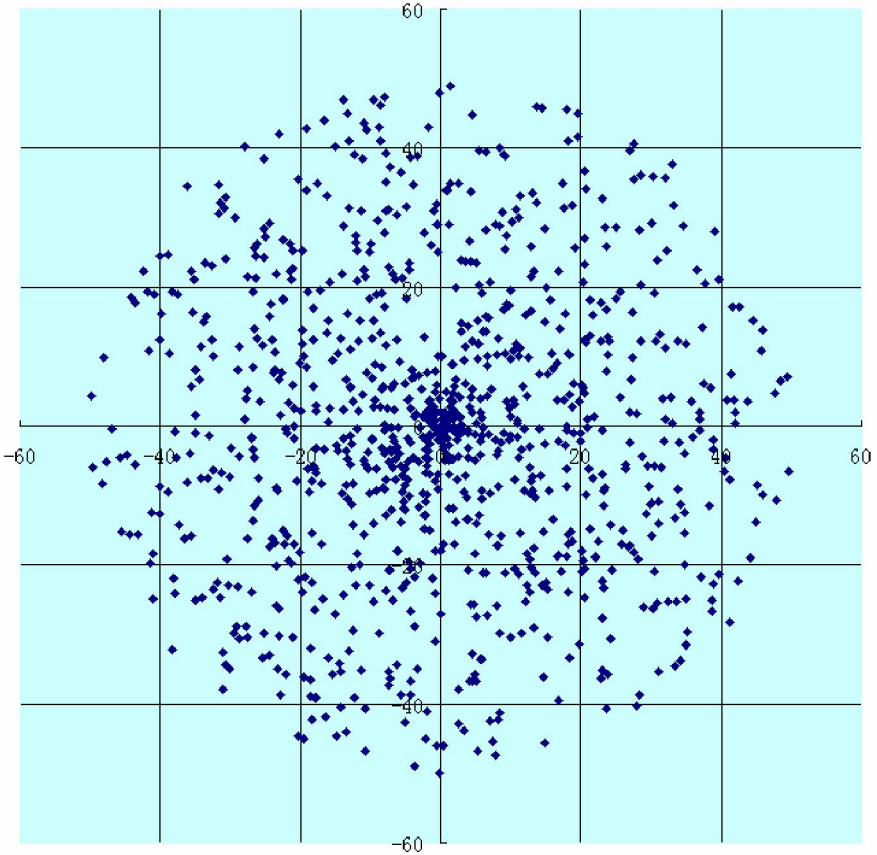
$$p = C_{34}^{14} \cdot 4^{14} \Big/ C_{136}^{14} = \frac{[21 \sim 34] \cdot 4^{14}}{[123 \sim 136]} \approx 8.79\%$$

圆内均匀取点

- 给定定点$O(x_0,y_0)$和半径r，使得二维随机点(x,y)等概率落在圆内。
- 分析
  - 因为均匀分布的数据是具有平移不变性，生成半径为r，定点为圆心的随机数(x_1,y_1)，然后平移得到(x_1+x_0,y_1+y_0)即可。
  - 直接使用$x=r*\cos\theta$，$y=r*\sin\theta$是否可以呢？
    - 具体试验一下。

圆内均匀取点代码与效果

```
int rand50()
{
    return rand() % 100 - 50;
}

int _tmain(int argc, _TCHAR* argv[])
{
    ofstream oFile;
    oFile.open(_T("D:\\rand.txt"));
    double r,theta;
    double x,y;
    for(int i = 0; i < 1000; i++)
    {
        r = rand50();
        theta = rand();
        x = r*cos(theta);
        y = r*sin(theta);
        oFile << x << '\t' << y << '\n';
    }
    oFile.close();
    return 0;
}
```



代码与效果

- 显然上述做法是不对的。但可以使用二维随机点的做法，若落在圆外，则重新生成点。结果如下。

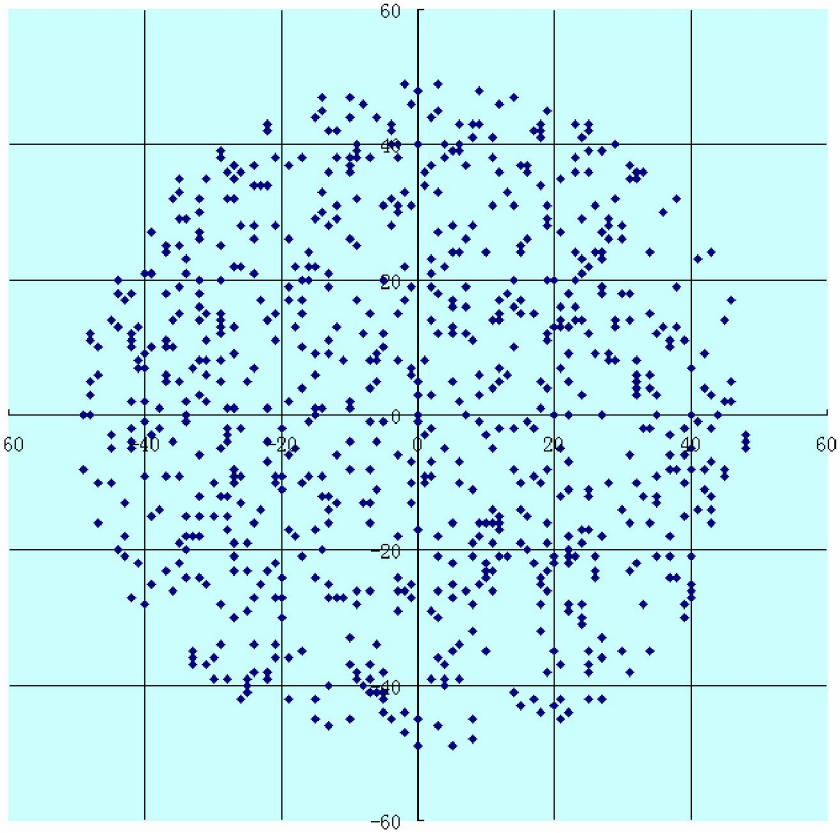
复习：rand10问题

```
int rand10()
{
    int a1, a2, r;
    do
    {
        a1 = rand7() - 1;
        a2 = rand7() - 1;
        r = a1 * 7 + a2;
    } while (r >= 40);
    return r / 4 + 1;
}
```

- 已知有个rand7()的函数，返回1到7随机自然数，让利用这个rand7()构造rand10() 随机1~10。
- 解：因为rand7仅能返回1~7的数字，少于rand10的数目。因此，多调用一次，从而得到49种组合。超过10的整数倍部分，直接丢弃。

代码与效果

```
int rand50()
{
    return rand() % 100 - 50;
}

int _tmain(int argc, _TCHAR* argv[])
{
    ofstream oFile;
    oFile.open(_T("D:\\rand.txt"));
    int x,y;
    for(int i = 0; i < 1000; i++)
    {
        x = rand50();
        y = rand50();
        if(x*x + y*y < 2500)
            oFile << x << '\t' << y << '\n';
    }
    oFile.close();
    return 0;
}
```



思想分析

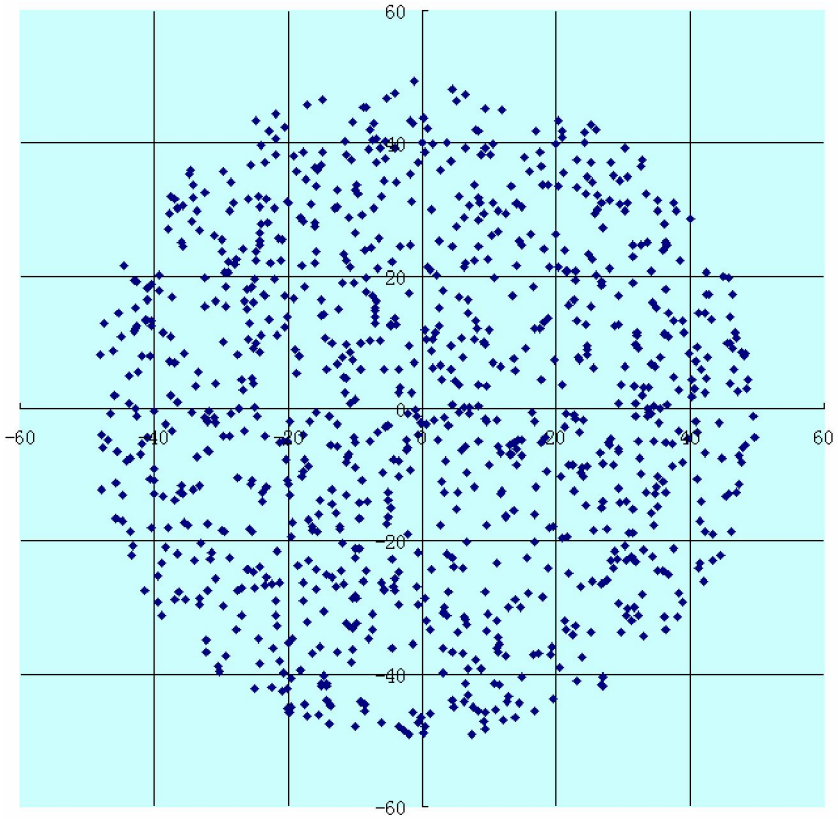
- 不是每次生成随机数都能退出该算法
  - 有一定的接受率。
  - 思考：
    - 以多大的概率1次退出：接受率是多少？
    - 得到随机数的需要的平均次数(期望)是多少？
    - 利用该方法计算圆周率？
- 这个做法简洁、有效，值得推荐；
  - 许多相关问题，往往可以如此解决。

圆内均匀取点的1次成功算法(朴素)

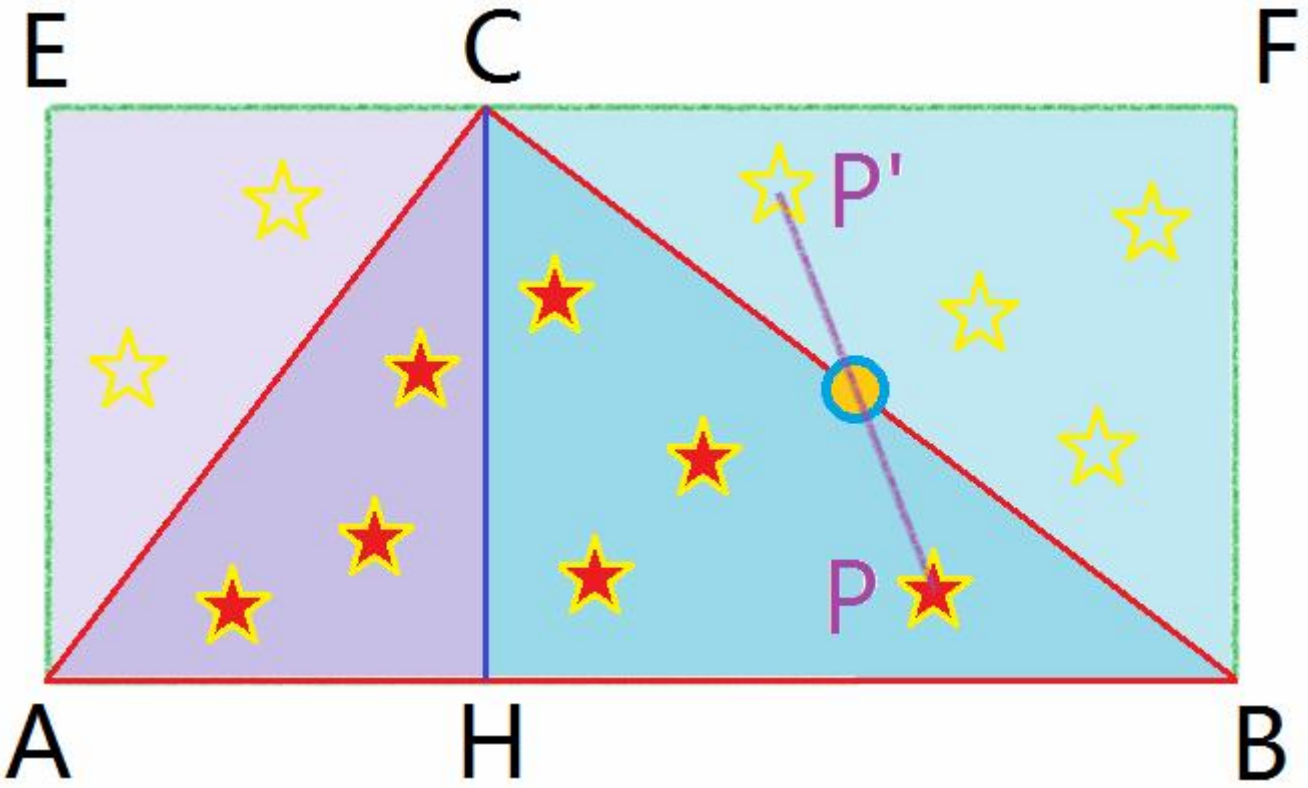
- 问题分析：把随机点看做面积很小的区域，圆内均匀取点意味着随机点P的面积与圆的面积成正比。
- $S_p=kS$
- $S=\pi r^2$，与半径的平方成正比
- 从而，$S_p(r)=k\pi r^2$
- 将均匀生成的随机数x取平方根赋值给r；则$S_p(r)$即为均匀分布。
- 同时，是与角度θ无关的，即：取均匀分布的随机数θ作为旋转角即可。

代码与效果

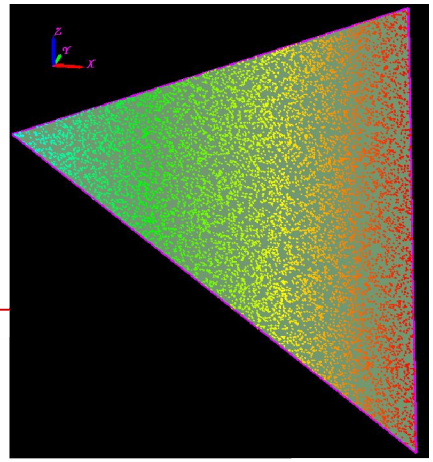
```
double rand2500()
{
    return rand() % 2500;
}

int _tmain(int argc, _TCHAR* argv[])
{
    ofstream oFile;
    oFile.open(_T("D:\\rand.txt"));
    double r,theta;
    double x,y;
    for(int i = 0; i < 1000; i++)
    {
        r = sqrt(rand2500());
        theta = rand();
        x = r*cos(theta);
        y = r*sin(theta);
        oFile << x << '\t' << y << '\n';
    }
    oFile.close();
    return 0;
}
```

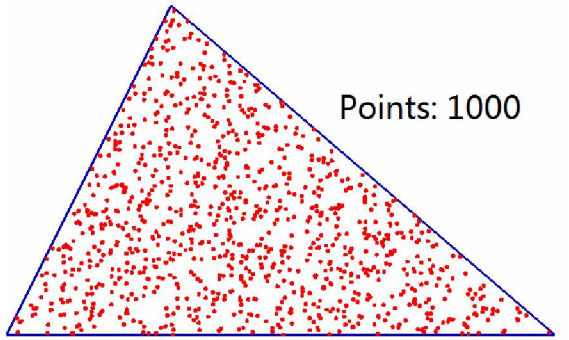


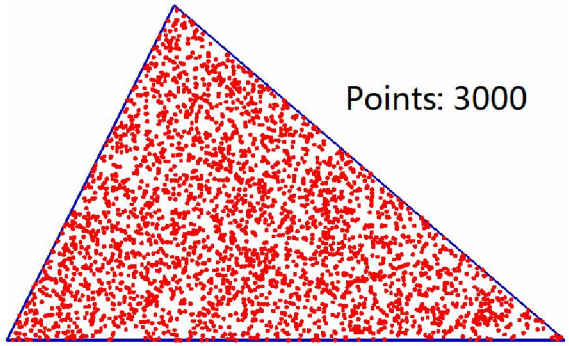
思考：将圆域换成三角形呢？



代码与效果








```
void CRandomTriangle::Random2(int nSize)
{
    CalcRotate();
    m_nSize = nSize;
    if(m_pRandomPoint)
        delete[] m_pRandomPoint;
    m_pRandomPoint = new CDelPoint[nSize];
    CDelPoint pt;
    for(int i = 0; i < nSize; i++)
    {
        pt.RandomInRectangle(m_ptExtend, m_ptHeight);
        if(m_tsBig.IsIn(pt))
        {
            pt += m_ptBase;
            m_pRandomPoint[i] = pt;
        }
        else if(m_tsLeft.IsIn(pt))
        {
            CDelPoint::MirrorPoint(pt, m_ptLeft0);
            pt += m_ptBase;
            m_pRandomPoint[i] = pt;
        }
        else if(m_tsRight.IsIn(pt))
        {
            CDelPoint::MirrorPoint(pt, m_ptRight0);
            pt += m_ptBase;
            m_pRandomPoint[i] = pt;
        }
    }
    CDelPoint::Save(m_pRandomPoint, m_nSize, _T("D:\\random.pt"), 0);
}
```

进一步思考

- 由于三点共面，所以三角形内的所有点必然在某平面上，因此，上述算法能够方便的推广到三维空间。
- 问题：请设计多边形内随机取点算法。
  - 圆内取点的思想：计算多边形的外包围矩形盒，生成外包围盒内的二维点，若点在多边形内，则退出；否则，继续探测。
  - 将多边形剖分成三角形集合，调用三角形内均匀取点算法。
- 算法2思路：
  - 按照面积为权重，选择某个三角形；
  - 生成该三角形内的随机点。
- 拓展
  - 每首歌有不同的分值，设计算法，根据分值随机推荐歌曲。
  - 如何将多边形快速剖分成三角形？注：Delaunay三角剖分

分值推荐

- 假定歌曲库中N首歌，每首歌给定一个整数分数。现在要求从N首歌中随机选择若干首推荐给用户，要求推荐的这些歌是和其分数作为正比的。
- 给定整数数组A[0…N-1]，按照A[i]的值作为权值随机取数。
  - 该问题在主题模型LDA中会再次遇到。

分析

- 根据权值A[0…N-1]计算累积权值B[0…N-1]
  - $B_0=A_0$
  - $B_i=B_{i-1}+A_i$
- 区间$[B_{i-1}, B_i)$对应元素A_i

Code

```
int RandSong(const int* song, int size)
{
    int i;
    int* pCumulate = new int[size];//i的范围：[i-1,i)
    pCumulate[0] = song[0];
    for(i = 1; i < size; i++)
        pCumulate[i] = pCumulate[i-1] + song[i];

    int nRec = rand() % pCumulate[size-1];
    int low = 0;
    int high = size-1;
    int mid;
    int nSong = -1;
    while(low < high)
    {
        mid = (low + high) / 2;
        if(nRec < pCumulate[mid])
            high = mid;
        else if(nRec > pCumulate[mid])
            low = mid + 1;
        else
        {
            nSong = mid+1;
            break;
        }
    }
    if(nSong == -1)
        nSong = low;
    delete[] pCumulate;
    return nSong;
}
```

另外的思路

- 计算所有歌的权值和sum，每首歌的权值除以sum，认为是各自的概率P[0…N-1]；
- 等概率选择nSong ∈ [0,N-1)
- 生成p ∈ [0,1]，若p < P[nSong]，则选择nSong，否则，计算随机生成新的nSong，继续探测。

Code

```
int RandSong2(const int* song, int size)
{
    int nSum = song[0];
    for(int i = 1; i < size; i++)
        nSum += song[i];

    bool bFind = false;
    int nCandidate = 0;
    while(!bFind)
    {
        nCandidate = rand() % size; //候选
        if(rand() % nSum < song[nCandidate])
        {
            bFind = true;
            break;
        }
    }
    return nCandidate;
}
```

试验结果

- 初始值： song = {43,63,43,89,322,2,5,32}
- 真实概率：
  - 0.0718 0.105 0.0718 0.149 0.538 0.00334 0.00835 0.0534
- 算法1
  - 10^3 ： 0.073 0.096 0.077 0.152 0.536 0.004 0.013 0.049
  - 10^4： 0.0758 0.107 0.0723 0.143 0.54 0.0028 0.0087 0.0505
- 算法2
  - 10^3： 0.076 0.095 0.084 0.137 0.539 0.003 0.008 0.058
  - 10^4： 0.0719 0.101 0.0726 0.144 0.542 0.0025 0.0093 0.0567

思考

- 给定N个数，设计算法，输出随机排列的一个结果。
  - 要求：输出任何一个排列的概率是相同的。
  - C++ STL std::random_shuffle

算法1

```
int Random(int N)    //[0,N]
{
    return rand() % (N+1);
}

void RandomShuffle(int* a, int size)
{
    int j;
    for(int i = 1; i < size; i++)    //待生成倒数第i个数
    {
        j = Random(size-i); //[0,size-i]
        swap(a[j],a[size-i]);
    }
}

int _tmain(int argc, _TCHAR* argv[])
{
    const int N = 10;
    int a[N];
    for(int i = 0; i < N; i++)
        a[i] = i+1;
    RandomShuffle(a, N);
    Print(a, N);
    return 0;
}
```

算法2

```
int Random2(int a, int b)
{
    return rand() % (b-a+1) + a;
}

void RandomShuffle2(int* a, int size)
{
    int j;
    for(int i = 0; i < size-1; i++) //i为待生成的第几个数
    {
        j = Random2(i, size-1); //[i,size-1]
        swap(a[i],a[j]);
    }
}
```

金钗赠诗问题

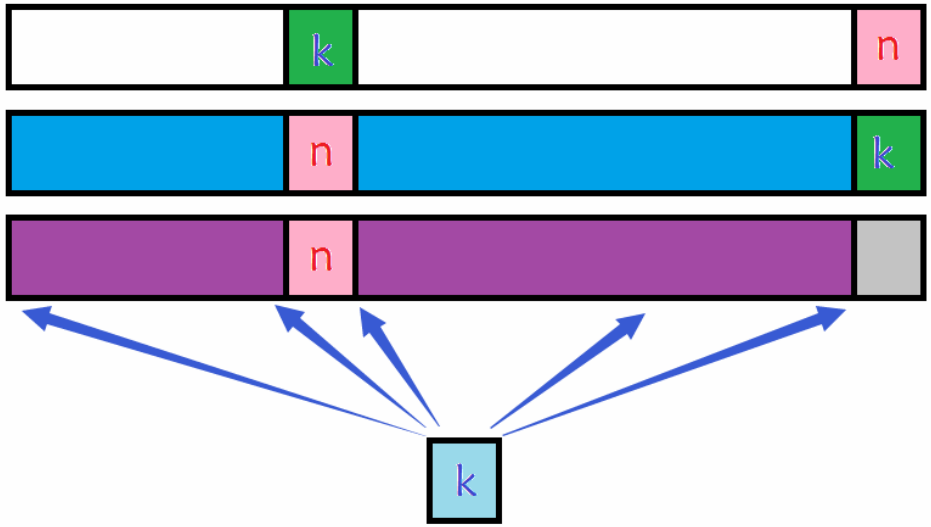
- 赛诗会后，十二金钗待奔前程。分别宴上，12人各写一首诗放入宝囊。大家任取，若取到自己的诗，则再取一首并放回自己的诗。12人都拿到别人的诗作算一种分配。问：共有多少种不同的分配？

问题的由来

- 本质即给定n个人写n首诗，要求赠给其他人，共有多少种分配方法。
- 一般提法：1到n的全排列中，第i个数不是i的排列共有多少种？
  - 最早是由丹尼尔·伯努利(Danid Bernoulli)提出的“错位排列”问题。

问题分析

- 假定n个数的错位排列数目为dp[n]
- 先考察数字n的放置方法：显然，n可以放在从1到n-1的某个位置，共n-1种方法；假定放在了第k位。
- 对于数字k：
  - 要么放置在第n位，
  - 要么不放置在第n位。
  - 下面分别讨论

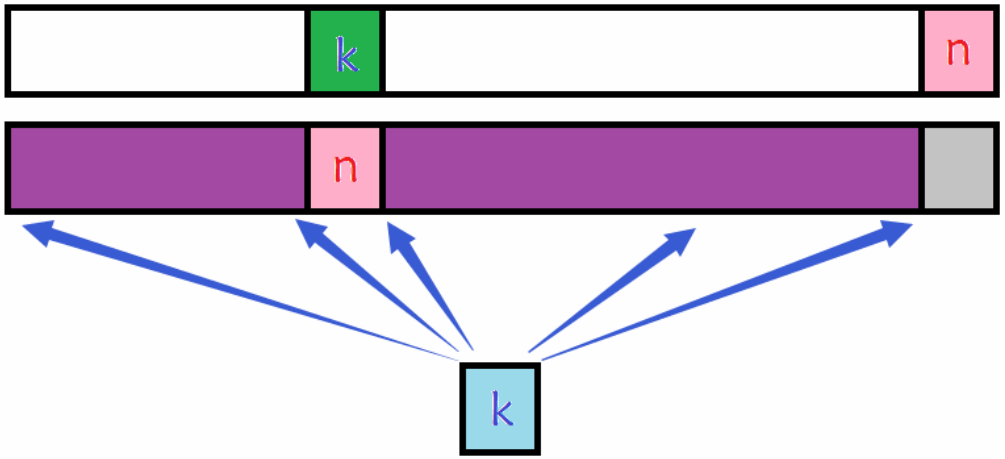


数字k放置在第n位

- 相当于数字k和数字n交互位置后，其他n-2个数字做错位排列，因此有dp[n-2]种方法。

数字k不放置在第n位

- 将数字k暂时更名为n(这是可以做到的：因为真正的n已经放在第k位上，真正的n不再考虑之列)，现在需要将1到k-1以及k+1到n这n-1个数放置在相应位置上，要求数字和位置不相同！显然是n-1个数的错位排列，有dp[n-1]种方法。



错位排列递推公式

- 综上，dp[n]=(n-1)*(dp[n-1]+dp[n-2])
- 初值：
  - 只有1个数字，错位排列不存在，dp[1]=0；
  - 只有2个数字，错位排列即交换排列，dp[2]=1；

$$a(n)=\begin{cases}(n-1)\cdot\left[a(n-1)+a(n-2)\right] & n>2\\ 1 & n=2\\ 0 & n=1\end{cases}$$

错位排列的通项公式

- 使用基本的加法和乘法原理，能够得到错位公式的通项形式：

$$S(n)=P_n^n-C_n^1P_{n-1}^{n-1}+-C_n^2P_{n-2}^{n-2}+\cdots+(-1)^n-C_n^nP_0^0$$

- 或等价形式：

$$S(n)=n!\left(1-\frac{1}{1!}+\frac{1}{2!}-\frac{1}{3!}+\frac{1}{4!}+\cdots+(-1)^n\frac{1}{n!}\right)$$

附：简洁递归公式 $f(n)=nf(n-1)+(-1)^n$

□ 原始公式n→n-1：

$$f(n)=(n-1)\cdot[f(n-1)+f(n-2)]$$
$$\Rightarrow f(n-1)=(n-2)\cdot[f(n-2)+f(n-3)]$$

□ 变换：

$$f(n)=(n-1)\cdot[f(n-1)+f(n-2)]$$
$$\Rightarrow f(n)-nf(n-1)=-[f(n-1)-(n-1)\cdot f(n-2)]$$
$$=-[\underline{(n-2)\cdot[f(n-2)+f(n-3)]}-(n-1)\cdot f(n-2)]$$
$$=-[-f(n-2)+(n-2)\cdot f(n-3)]$$
$$=+[f(n-2)-(n-2)\cdot f(n-3)]$$

□ 递归关系：

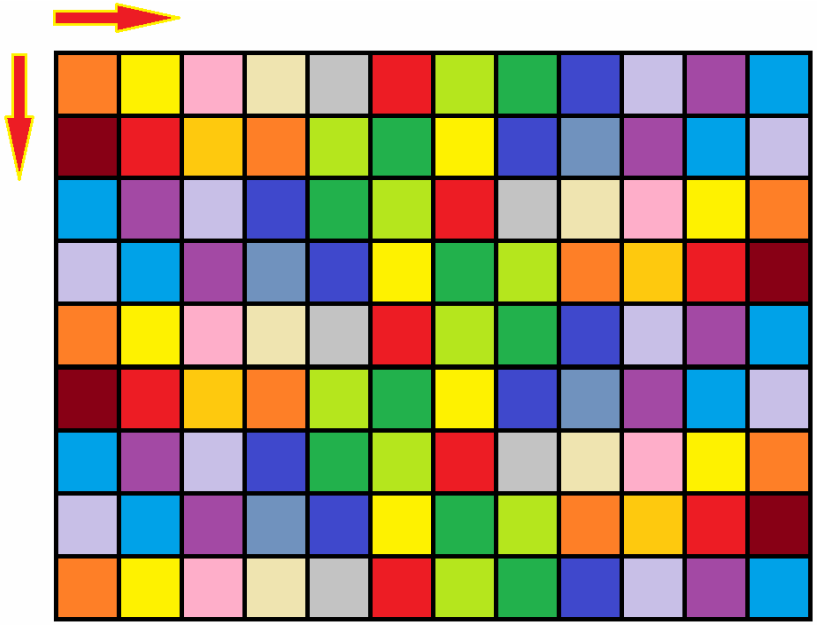
$$f(n)-nf(n-1)$$
$$=-[f(n-1)-(n-1)\cdot f(n-2)]$$
$$=+[f(n-2)-(n-2)\cdot f(n-3)]$$
$$=-[f(n-3)-(n-3)\cdot f(n-4)]$$
$$=\cdots\cdots$$
$$=(-1)^{n-2}[f(2)-2\cdot f(1)]$$
$$=(-1)^{n-2}=(-1)^n$$

思考题

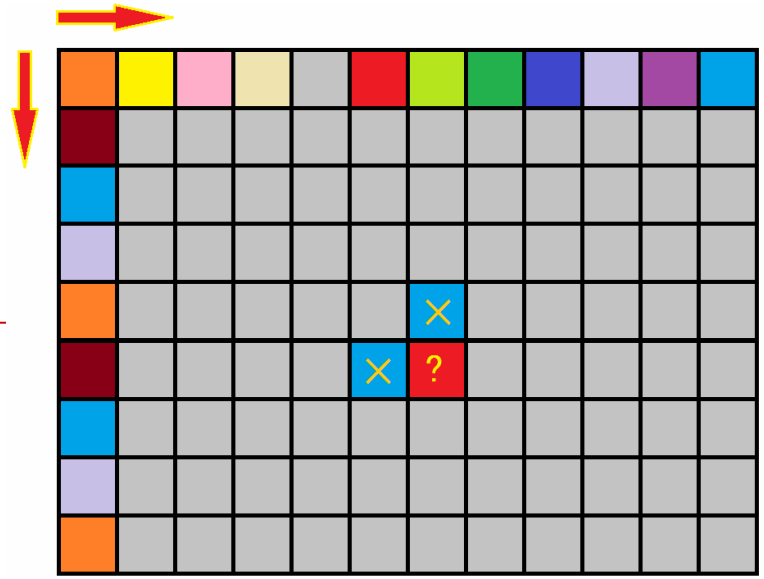
- 毕业典礼后，某宿舍三位同学把自己的毕业帽扔了，随后每个人随机地拾起帽子，试计算三个人中没有人选到自己原来带的帽子的概率。

走棋盘/格子取数

- 给定m*n的矩阵，每个位置是一个非负整数，从左上角开始，每次只能朝右和下走，走到右下角，求总和最小的路径。

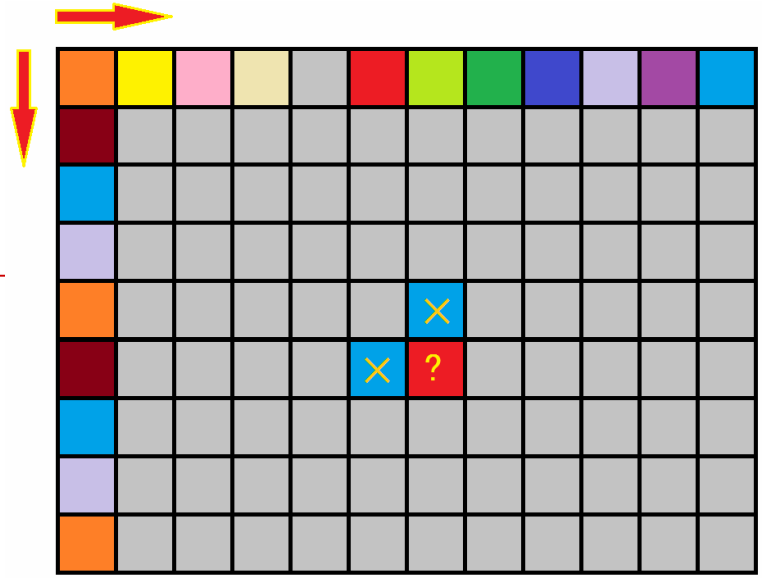


状态转移方程



- 走的方向决定了同一个格子不会经过两次。
  - 若当前位于(x,y)处，它来自于哪些格子呢？
  - dp[0,0]=a[0,0] / 第一行(列)累积
  - dp[x,y] = min(dp[x-1,y]+a[x,y],dp[x,y-1]+a[x,y])
  - 即：dp[x,y] = min(dp[x-1,y],dp[x,y-1]) +a[x,y]
- 思考：若将上述问题改成“求从左上到右下的最大路径”呢？

状态转移方程



- 状态转移方程：

$$\begin{cases} dp(i,0)=\sum_{k=0}^{i} chess[k][0] \\ dp(0,j)=\sum_{k=0}^{j} chess[0][k] \\ dp(i,j)=\min(dp(i-1,j),dp(i,j-1))+chess[i][j] \end{cases}$$

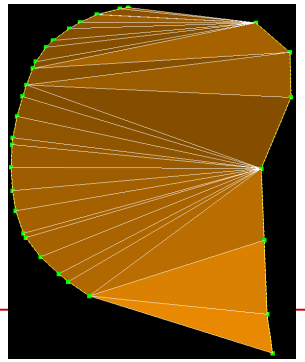
- 滚动数组：

$$\begin{cases} dp(j)=\sum_{k=0}^{j} chess[0][k] \\ dp(j)=\min(dp(j),dp(j-1))+chess[i][j] \end{cases}$$

Code

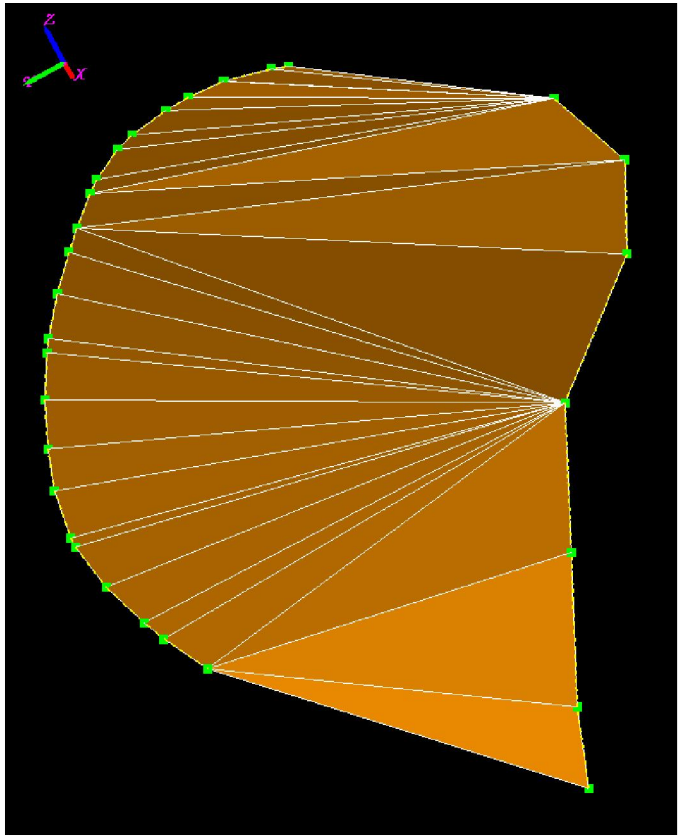
```
int MinPath(vector<vector<int> >& chess, int M, int N)
{
    vector<int> pathLength(N);
    int i,j;

    //初始化
    pathLength[0] = chess[0][0];
    for(j = 1; j < N; j++)
        pathLength[j] = pathLength[j-1] + chess[0][j];

    //依次计算每行
    for(i = 1; i < M; i++)
    {
        pathLength[0] += chess[i][0];
        for(j = 1; j < N; j++)
        {
            if(pathLength[j-1] < pathLength[j])
                pathLength[j] = pathLength[j-1] + chess[i][j];
            else
                pathLength[j] += chess[i][j];
        }
    }
    return pathLength[N-1];
}

int _tmain(int argc, _TCHAR* argv[])
{
    const int M = 10;
    const int N = 8;
    vector<vector<int> > chess(M, vector<int>(N));

    //初始化棋盘：(随机给定)
    int i,j;
    for(i = 0; i < M; i++)
    {
        for(j = 0; j < N; j++)
            chess[i][j] = rand() % 100;
    }
    cout << MinPath(chess, M, N) << endl;
    return 0;
}
```

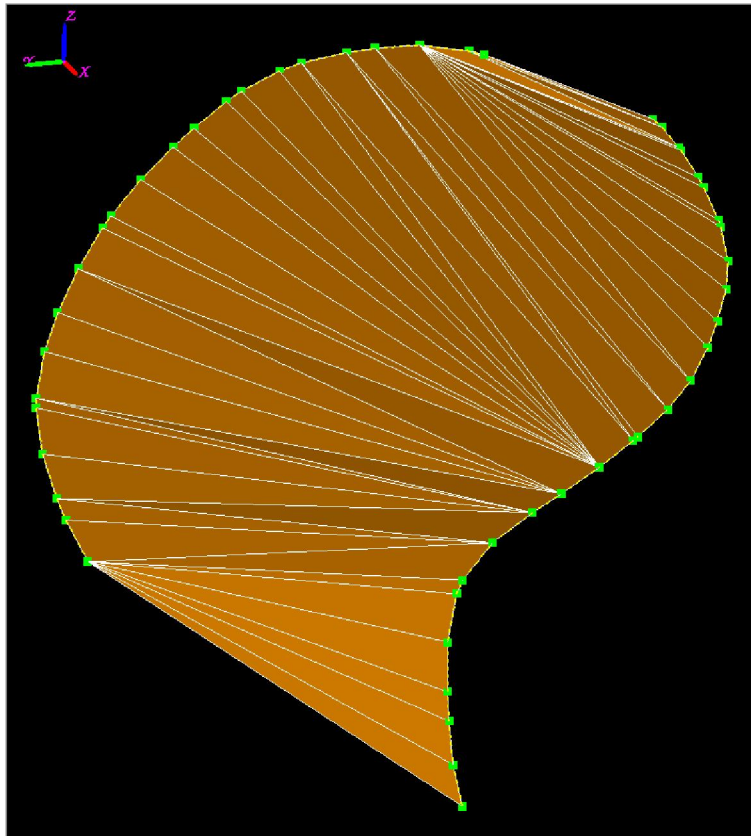
实践应用



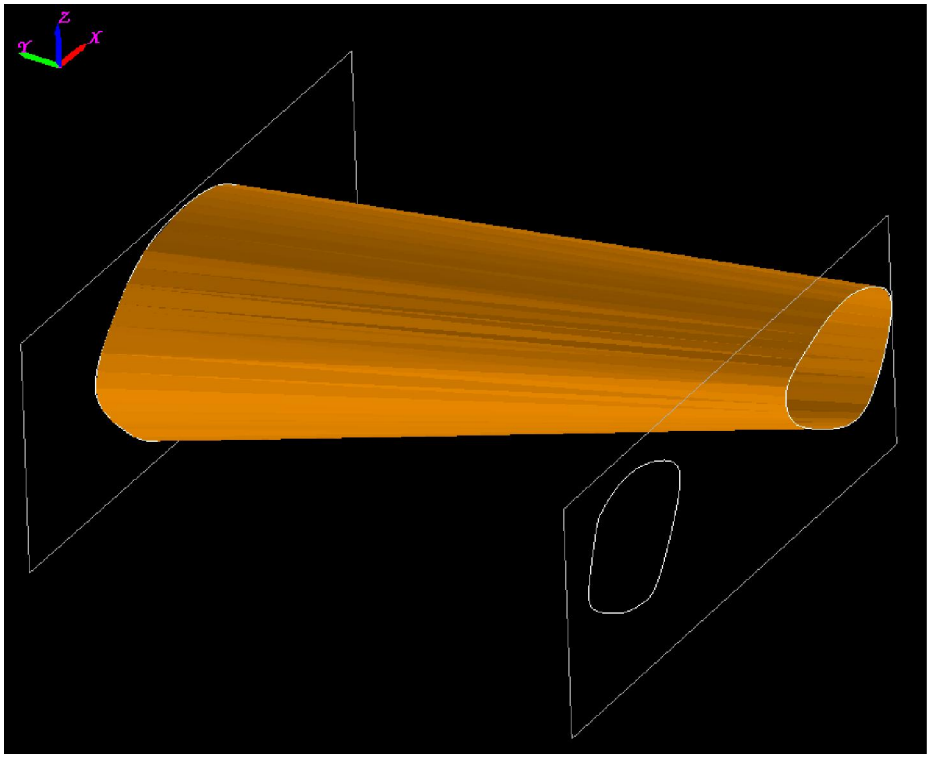
- 给定空间两条曲线，设计经过这两条曲线的曲面，使得该曲面是“最合理”的。
  - 探讨如下方案：两条曲线上的点序列，记做P1[0…N-1]和P2[0…M-1]，计算P1[i]和P2[j]的距离，从而得到二维表格T[N][M]。从T[0][0]开始出发，只能向右和向下走(曲面不能自相交)，从(0,0)到(N-1,M-1)的最短路径，就是曲面的一种“合理连接”。

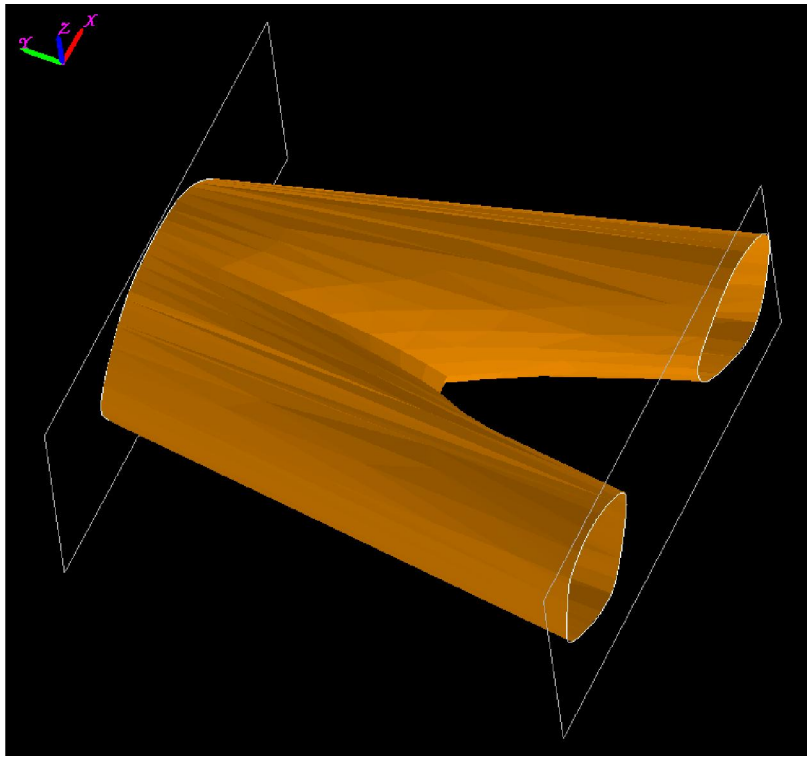
实践：GIS中的应用





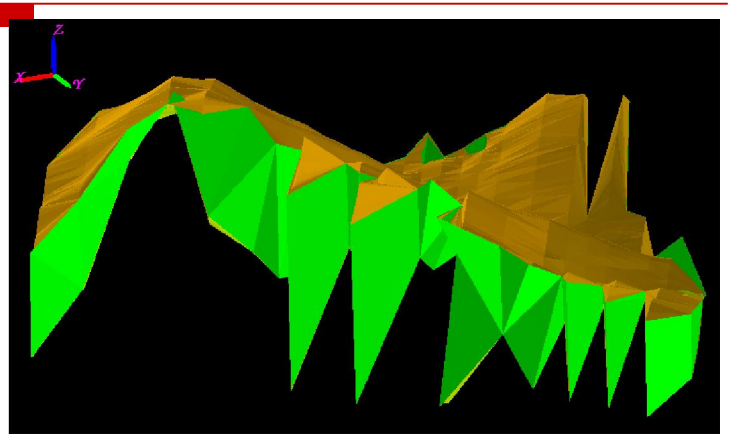
如果三维曲线是封闭线…

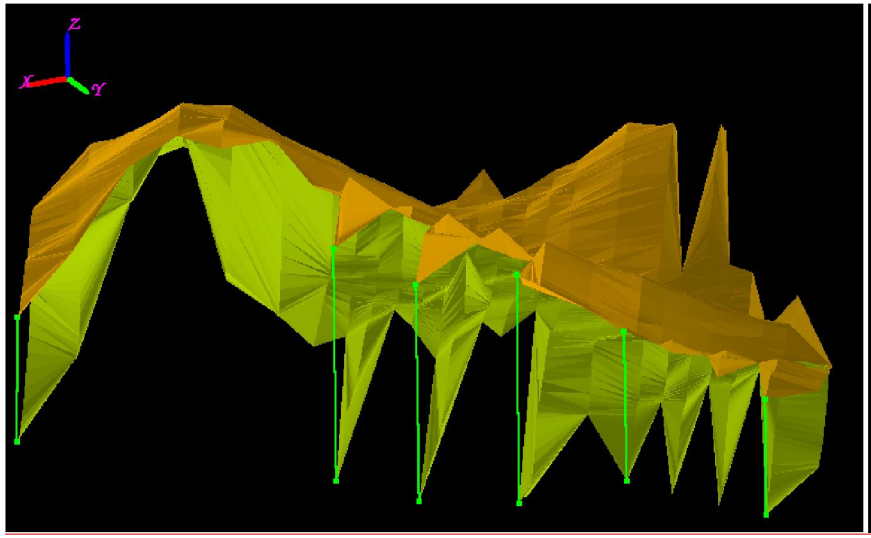


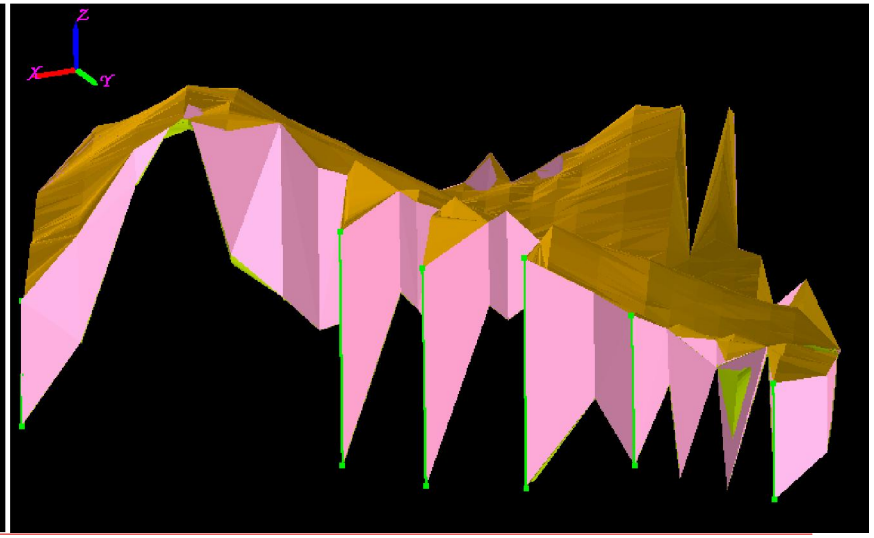


经过引导线的曲面——带约束的走棋盘

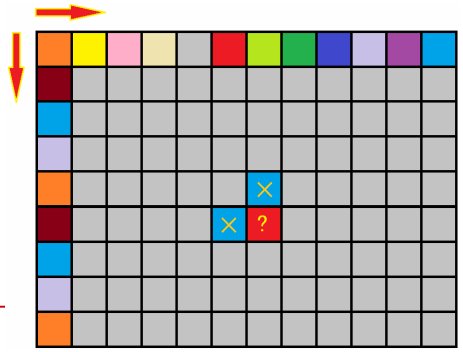
- 右上：未使用引导线
- 左下：输入的引导线
- 右下：过引导线的曲面







动态规划递推式可以得到？



- 令dp(x,y)为当前位于(x,y)时有多少种可行路径，则：dp(x,y)=dp(x-1,y)+dp(x,y-1)

$$dp(x,y)=dp(x-1,y)+dp(x,y-1)$$

$$\xrightarrow{\text{增加“两个坐标值加和”}} dp(x+y,x,y)=dp(x+y-1,x-1,y)+dp(x+y-1,x,y-1)$$

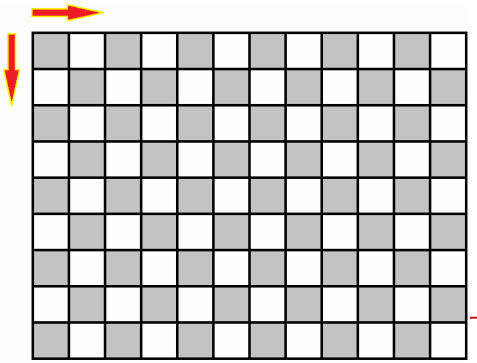
$$\xrightarrow{\text{删除最后一维}} dp(x+y,x)=dp(x+y-1,x-1)+dp(x+y-1,x)$$

$$\xrightarrow{\text{令}t=x+y} dp(t,x)=dp(t-1,x-1)+dp(t-1,x)$$

$$\xrightarrow{\text{换个表达方式}} C_t^x=C_{t-1}^{x-1}+C_{t-1}^x$$

$$\xrightarrow{\text{令}n=t,m=x} C_n^m=C_{n-1}^{m-1}+C_{n-1}^m$$

走棋盘数目

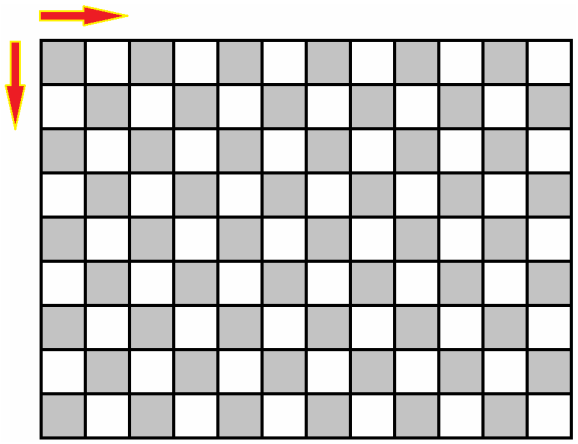


- 给定m×n的矩阵，从左上角开始行走，每次只能朝右和下走，走到右下角，求一共有多少种走法？
  - 一共要走m+n–2步，其中(m–1)步向右，(n-1)步向下。即：在m+n–2个步数中，选择m–1个作为“右行”。

$$N = C_{m+n-2}^{m-1} = C_{m+n-2}^{n-1}$$

带陷阱的走棋盘

- 给定m×n的棋盘0/1矩阵，将某机器人放置于棋盘左上角，机器人每一步只能往右或者往下移动。棋盘中有些格子是禁止机器人踏入的，请计算从左上角到右下角共有多少条路径。



状态转移方程

- dp[i][j]表示从起点到(i,j)的路径条数。
- 只能从左边或者上边进入一个格子
- 如果(i,j)被占用
  - dp[i][j]=0
- 如果(i,j)不被占用
  - dp[i][j]=dp[i-1][j]+dp[i][j–1]

Code

```
int _tmain(int argc, _TCHAR* argv[])
{
    const int M = 4;
    const int N = 5;
    vector<vector<bool> > chess(M, vector<bool>(N));

    int i,j;
    for(i = 0; i < M; i++)
    {
        for(j = 0; j < N; j++)
            chess[i][j] = ((rand() % 5) != 0);
    }
    cout << PathNumber(chess) << endl;
    return 0;
}
```

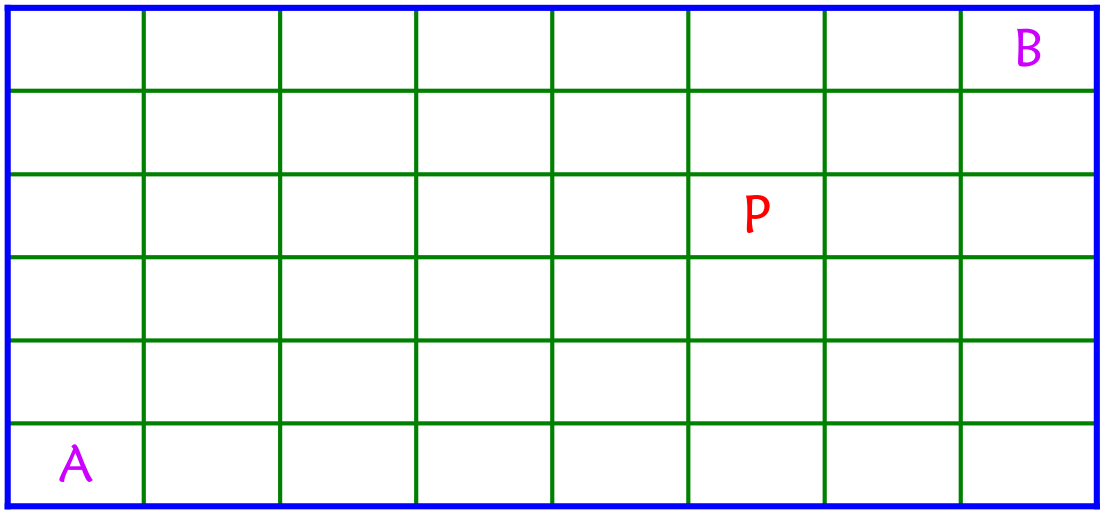
```
int PathNumber(vector<vector<bool> >& chess)
{
    int M = (int)chess.size();        //行
    int N = (int)chess[0].size();     //列
    vector<int> pathNumber(N, 0);
    int i,j;
    pathNumber[0] = chess[0][0] ? 1 : 0;
    for(j = 1; j < N; j++)
    {
        //当前未阻断，并且前一个值可达
        if(chess[0][j] && (pathNumber[j-1] == 1))
            pathNumber[j] = 1;
    }

    for(i = 1; i < M; i++)
    {
        if(!chess[i][0])
            pathNumber[0] = 0;
        for(j = 1; j < N; j++)
        {
            if(!chess[i][j])      //当前被阻断
                pathNumber[j] = 0;
            else                  //Cur=Left+Top
                pathNumber[j] += pathNumber[j-1];
        }
    }
    return pathNumber[N-1];
}
```

思考题

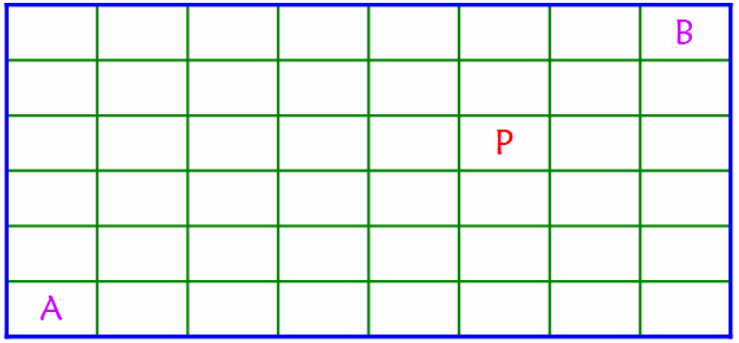
- 如何将上述代码使用滚动数组的方式降低空间复杂度？

陷阱走棋盘应用

- 在8×6的矩阵中，每次只能向上或向右移动一格，并且不能经过P。试计算从A移动到B一共有多少种走法。

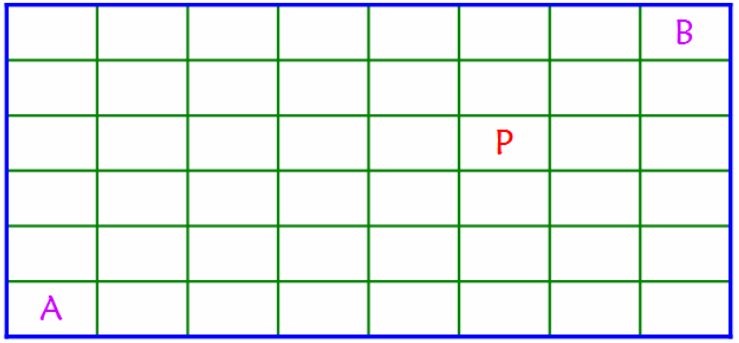


解题过程



- 从A到B共需要移动12步，其中7步向右，5步向上，可行走法数目为 $C_{12}^{5}=792$
- 从A到P共需要8步，其中5步向右，3步向上，可行走法数目为 $C_{8}^{5}=56$
- 从P到B共需要4步，其中2步向右，2步向上，可行走法数目为 $C_{4}^{2}=6$
- 则，从A到B经过P的路线有56*6=336种；
- 从A到B不经过P的路线有792-336=456种。

方格的可行路径数目



1	6	21	56	126	196	294	456
1	5	15	35	70	70	98	162
1	4	10	20	35	0	28	64
1	3	6	10	15	21	28	36
1	2	3	4	5	6	7	8
1	1	1	1	1	1	1	1

实数的整数次幂

- 给定实数x和整数n，求x^n。
- 分析：令pow(x,n)= x^n，如果能够求出y=pow(x,n/2)，只需要返回y*y即可，节省一半的时间。因此，可以递归下去。
  - 时间复杂度O(NlogN)
  - 需要考虑的：如果n是奇数呢？
  - 如果n是负数呢？

Code

```
double Pow(double x, int n)
{
    if(n == 0)
        return 1;
    if(n == 1)
        return x;
    if(n == 2)
        return x*x;
    double p = Pow(x, n/2);
    p *= p;
    return (n % 2 == 0) ? p : p*x;
}

double Power(double x, int n)
{
    if(n < 0)
        return 1/Pow(x, -n);
    return Pow(x, n);
}
int _tmain(int argc, _TCHAR* argv[])
{
    cout << Power(1.01, 365) << endl;
    return 0;
}
```

Code

```
double Pow(double x, int n)
{
    if(n == 0)
        return 1;
    if(n == 1)
        return x;
    if(n == 2)
        return x*x;
    double p = Pow(x, n/2);
    p *= p;
    return (n % 2 == 0) ? p : p*x;
}

double Power(double x, int n)
{
    if(n < 0)
        return 1/Pow(x, -n);
    return Pow(x, n);
}
```

矩阵的乘法

- A为m×s阶的矩阵，B为s×n阶的矩阵，那么，C=A×B是m×n阶的矩阵，其中，

$$c_{ij} = \sum_{k=1}^{s} a_{ik} b_{kj}$$

- 根据定义来计算 C=A×B，需要m*n*s次乘法。
  - 即：若A、B都是n阶方阵，C的计算时间复杂度为$O(n^3)$
  - 问：可否设计更快的算法？

分治

- 矩阵分块

$$\begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \cdot \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix} = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

$$\begin{cases} C_{11} = A_{11}B_{11} + A_{12}B_{21} \\ C_{12} = A_{11}B_{12} + A_{12}B_{22} \\ C_{21} = A_{21}B_{11} + A_{22}B_{21} \\ C_{22} = A_{21}B_{12} + A_{22}B_{22} \end{cases}$$

- 按照定义：计算n/2阶矩阵的乘积：$O(n^3/8)$
- 这里需要计算8个：总时间复杂度：$O(n^3)$
  - 没有任何效果。

Strassen矩阵乘法：由8到7

- 目标

$$\begin{cases} C_{11} = A_{11}B_{11} + A_{12}B_{21} \\ C_{12} = A_{11}B_{12} + A_{12}B_{22} \\ C_{21} = A_{21}B_{11} + A_{22}B_{21} \\ C_{22} = A_{21}B_{12} + A_{22}B_{22} \end{cases}$$

$$\begin{cases} P = (A_{11} + A_{22})(B_{11} + B_{22}) \\ Q = (A_{21} + A_{22})B_{11} \\ R = A_{11}(B_{12} - B_{22}) \\ S = A_{22}(B_{21} - B_{11}) \\ T = (A_{11} + A_{12})B_{22} \\ U = (A_{21} - A_{11})(B_{11} + B_{12}) \\ V = (A_{12} - A_{22})(B_{21} + B_{22}) \end{cases} \Longrightarrow \begin{cases} C_{11} = P + S - T + V \\ C_{12} = R + T \\ C_{21} = Q + S \\ C_{22} = P + S - Q + U \end{cases}$$

Strassen矩阵乘法的说明

- 时间复杂度降为$O(n^{\log 7})=O(n^{2.81})$
  - 理论意义：由定义出发直接得出的算法未必是最好的。
- Hopcroft与Kerr已经证明，两个2×2矩阵相乘必须要用7次乘法，如果需要进一步改进，应考虑3×3、4×4或者更高阶数的分块子矩阵。
  - 或采用并行或对n分治等其他策略。
- 当n很大时，实际效果比直接定义求解的$O(n^3)$好。
- 根据矩阵乘法的定义可知，C_{ij}只与A的第i行、B的第j列相关，在实践中若遇到大矩阵，应考虑并行计算。

$$c_{ij}=\sum_{k=1}^{s}a_{ik}b_{kj}$$

思考

- 将矩阵分治乘法的思想，用于两个大整数的乘法呢？
- 根据定义，两个大整数A、B相乘，应该遍历B从低到高的数字，依次与大整数A相乘，最后将这些值相加。
  - 时间复杂度$O(n^2)$。
  - 可否将A、B写成两个规模小一半的整数A1,A2,B1,B2，然后计算它们的积呢？

大整数乘法

- 取大整数x和y的长度较大者的一半，记为k，则有：

$$\begin{cases} x = x_1 M^k + x_0 \\ y = y_1 M^k + y_0 \end{cases}$$

$$\Rightarrow xy = \left(x_1 M^k + x_0\right)\left(y_1 M^k + y_0\right)$$

$$= x_1 y_1 M^{2k} + \left(x_1 y_0 + x_0 y_1\right) M^k + x_0 y_0$$

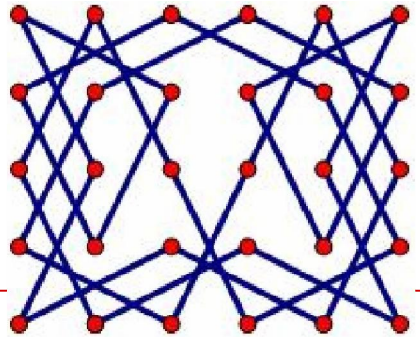
- 计算长度为n/2的两个数的积，需要乘法次数为$O(n^2/4)$，而上面的式子需要4次乘法，总时间复杂度为$O(n^2)$，没有效果。因此，需要考虑改进。

大整数乘法：Karatsuba算法

- 事实上：
$$\begin{cases} x = x_1 M^k + x_0 \\ y = y_1 M^k + y_0 \end{cases}$$
$$\Rightarrow xy = \left(x_1 M^k + x_0\right)\left(y_1 M^k + y_0\right)$$
$$xy = x_1 y_1 M^{2k} + \left(x_1 y_0 + x_0 y_1\right) M^k + x_0 y_0$$
$$= x_1 y_1 M^{2k} + \left(\left(x_1 + x_0\right)\left(y_1 + y_0\right) - x_1 y_1 + x_0 y_0\right) M^k + x_0 y_0$$

- 上式只需要3次乘法(配合若干次加法和移位)即可完成，时间复杂度为$O(n^{\log 3})=O(n^{1.585})$。

马踏棋盘



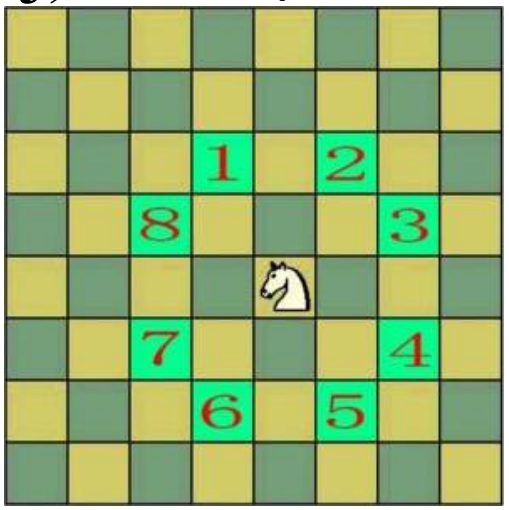
- 给定m×n的棋盘，将棋子“马”放在任意位置上，按照走棋规则将“马”移动，要求每个方格只能进入一次，最终使得“马”走遍棋盘的所有位置。
  - 如给定8×8的国际象棋棋盘(右)
  - 如给定8×9的中国象棋棋盘(左)

01	70	17	36	03	52	07	38	05
18	35	02	71	60	37	04	45	08
69	16	67	58	51	56	53	06	39
34	19	72	61	54	59	46	09	44
15	68	33	66	57	50	55	40	29
20	65	22	49	62	47	28	43	10
23	14	63	32	25	12	41	30	27
64	21	24	13	48	31	26	11	42

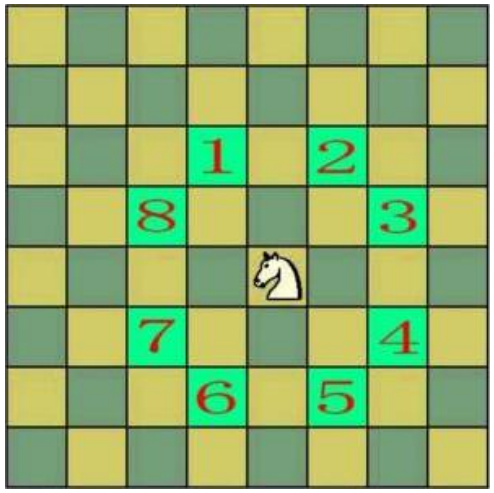
	23	48	35	10	21	56	33
49	36	11	22	55	34	9	20
12	1	24	47	40	57	32	63
25	50	37	54	43	46	19	8
2	13	44	41	58	39	62	31
51	26	53	38	45	42	7	18
14	3	28	59	16	5	30	61
27	52	15	4	29	60	17	6

问题分析

- 显然，如果从A点能够跳到B点，则从B点也能够跳到A点。所以，马的起始位置可以从任意一点开始，不妨从左上角开始。
- 若当前位置为(i,j)，则遍历(i,j)的八邻域，如果邻域尚未经过，则跳转。
  - 深度优先搜索



Code




```
int iHorse[] = {-2, -2, -1, +1, +2, +2, +1, -1};
int jHorse[] = {-1, +1, +2, +2, +1, -1, -2, -2};
int m = 8;
int n = 9;

bool CanJump(const vector<vector<int> >& chess, int i, int j)
{
    if((i < 0) || (i >= m) || (j < 0) || (j >= n))
        return false;
    return (chess[i][j] == 0);
}

bool Jump(vector<vector<int> >& chess, int i, int j, int step)
{
    if(step == m*n) //遍历结束
        return true;
    int iCur, jCur;
    for(int k = 0; k < 8; k++)
    {
        iCur = i + iHorse[k];
        jCur = j + jHorse[k];
        if(CanJump(chess, iCur, jCur))
        {
            chess[iCur][jCur] = step+1;
            if(Jump(chess, iCur, jCur, step+1))
                return true;
            chess[iCur][jCur] = 0;
        }
    }
    return false;
}
```

```
int _tmain(int argc, _TCHAR* argv[])
{
    vector<vector<int> > chess(m, vector<int>(n));
    chess[0][0] = 1;
    Jump(chess, 0, 0, 1);
    Print(chess);
    return 0;
}
```

启发式搜索

- 若棋盘规模较大，则在较深的棋位才能发现“无路可走”而不得不回溯。
- 贪心的启发式策略：
  - 最多情况下，每个棋位有8个后继。由于棋盘边界和已经遍历的原因，往往是少于8个的。
- 当前棋位可以跳转的后继棋位记为x个，这x个棋位的后继棋位数目记做$h_1h_2...h_x$，优先选择最小的h_i。
  - 策略：优先选择孙结点数目最少的那个子结点
  - 原因：孙结点最少的子结点，如果当前不跳转则最容易在后期无法跳转。

Code

```
int _tmain(int argc, _TCHAR* argv[])
{
    vector<vector<int> > chess(m, vector<int>(n));
    chess[0][0] = 1;
    Jump2(chess, 0, 0, 1);
    Print(chess);
    return 0;
}
```

```
typedef struct tagSHorse
{
    int nDirect;     //跳转方向
    int nValidStep; //有效步数

    bool operator < (const tagSHorse& horse)
    {
        return nValidStep < horse.nValidStep;
    }
}SHorse;
```

```
int GatherHorseDirect(SHorse* pHorse, int i, int j, const vector<vector<int> >& chess, bool bLast)
{
    int nHorse = 0;
    int iCur, jCur;
    for(int k = 0; k < 8; k++)
    {
        iCur = i + iHorse[k];
        jCur = j + jHorse[k];
        if(bLast)   //最后一步
        {
            if(CanJump(chess, iCur, jCur))
            {
                pHorse[nHorse].nValidStep = 1;
                pHorse[nHorse].nDirect = k;
                nHorse++;
                break;
            }
        }
        else    //正常情况
        {
            pHorse[nHorse].nValidStep = GetNextStep(chess, iCur, jCur);
            if(pHorse[nHorse].nValidStep != 0)
            {
                pHorse[nHorse].nDirect = k;
                nHorse++;
            }
        }
    }
    if(nHorse == 0)
        return 0;
    sort(pHorse, pHorse+nHorse);
    return nHorse;
}
```

```
bool Jump2(vector<vector<int> >& chess, int i, int j, int step)
{
    if(step == m*n) //遍历结束
        return true;
    SHorse pHorse[8];
    int nHorse = GatherHorseDirect(pHorse, i, j, chess, step == m*n-1);
    int iCur, jCur;
    int nDirect;
    step++;
    for(int k = 0; k < nHorse; k++)
    {
        nDirect = pHorse[k].nDirect;
        iCur = i + iHorse[nDirect];
        jCur = j + jHorse[nDirect];
        chess[iCur][jCur] = step;
        if(Jump2(chess, iCur, jCur, step))
            return true;
        chess[iCur][jCur] = 0;
    }
    return false;
}
```

Code

```
bool Jump2(vector<vector<int> >& chess, int i, int j, int step)
{
    if(step == m*n) //遍历结束
    {
        AddSolution(chess);
        return true;
    }
    SHorse pHorse[8];
    int nHorse = GatherHorseDirect(pHorse, i, j, chess, step == m*n-1);
    int iCur, jCur;
    int nDirect;
    step++;
    for(int k = 0; k < nHorse; k++)
    {
        nDirect = pHorse[k].nDirect;
        iCur = i + iHorse[nDirect];
        jCur = j + jHorse[nDirect];
        chess[iCur][jCur] = step;
        if(Jump2(chess, iCur, jCur, step))  //找到一个解
        {
            //return true;  //删去本行，则算法计算所有解
        }
        chess[iCur][jCur] = 0;
    }
    return false;
}
```

Code

```
int GatherHorseDirect(SHorse* pHorse, int i, int j, const vector<vector<int> >& chess, bool bLast)
{
    int nHorse = 0;
    int iCur, jCur;
    for(int k = 0; k < 8; k++)
    {
        iCur = i + iHorse[k];
        jCur = j + jHorse[k];
        if(bLast)   //最后一步
        {
            if(CanJump(chess, iCur, jCur))
            {
                pHorse[nHorse].nValidStep = 1;
                pHorse[nHorse].nDirect = k;
                nHorse++;
                break;
            }
        }
        else    //正常情况
        {
            pHorse[nHorse].nValidStep = GetNextStep(chess, iCur, jCur);
            if(pHorse[nHorse].nValidStep != 0)
            {
                pHorse[nHorse].nDirect = k;
                nHorse++;
            }
        }
    }
    if(nHorse == 0)
        return 0;
    sort(pHorse, pHorse+nHorse);
    return nHorse;
}
```

```
32
==============
1   20   5   12   9
6   13   10   17   4
19   2   15   8   11
14   7   18   3   16
==================
1   18   5   12   9
6   13   10   17   4
19   2   15   8   11
14   7   20   3   16
==================
1   16   5   12   9
6   13   10   19   4
17   2   15   8   11
14   7   18   3   20
==================
1   16   5   12   9
6   13   10   17   4
15   2   19   8   11
20   7   14   3   18
==================
1   16   5   12   9
6   19   10   15   4
17   2   13   8   11
20   7   18   3   14
==================
```

思考题：花开堪折直须折

- 婚介所登记了N位男孩和N位女孩，每个男孩都对N个女孩的喜欢程度做了排序，每个女孩都对N个男孩的喜欢程度做了排序。你作为月老，能否给出稳定的牵手方案？
  - 分析：每个男孩都对N个女孩做了排序，N个男孩的钟情度形成N×N的矩阵A；同时，N个女孩的钟情度形成N×N的矩阵B；
  - 如果男孩i和女孩a牵手，但男孩i对女孩b更喜欢，而女孩b的男友j拼不过男孩i，则没有力量阻碍男孩i和女孩b的私奔，这即是不稳定的。

稳定匹配问题

- 本题为著名的对稳定婚姻策略的Gale-Shapley算法，由于女孩选择对其表白的男孩的最优者，因此也称为延迟认可算法。
  - 可用于学生志愿填报(员工选择职位、毕业生选择单位)等“二部”问题。
  - 如果没有“有向边”，则该算法不适用。

我们在这里

- 问题答疑：http://www.xxwenda.com/
  - 烦请邀请visio或其他人回答问题

- 本课程QQ群：
  - 471632108(小象学院：机器学习群)

感谢大家！

恳请大家批评指正！